EPL and Esper
Contexts

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml

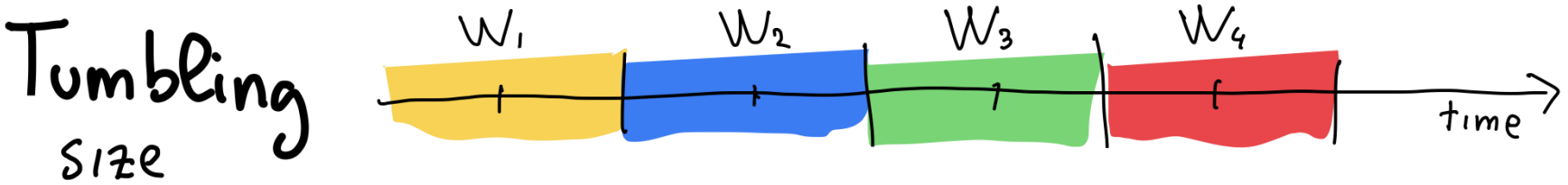
POLITECNICO
MILANO 1863

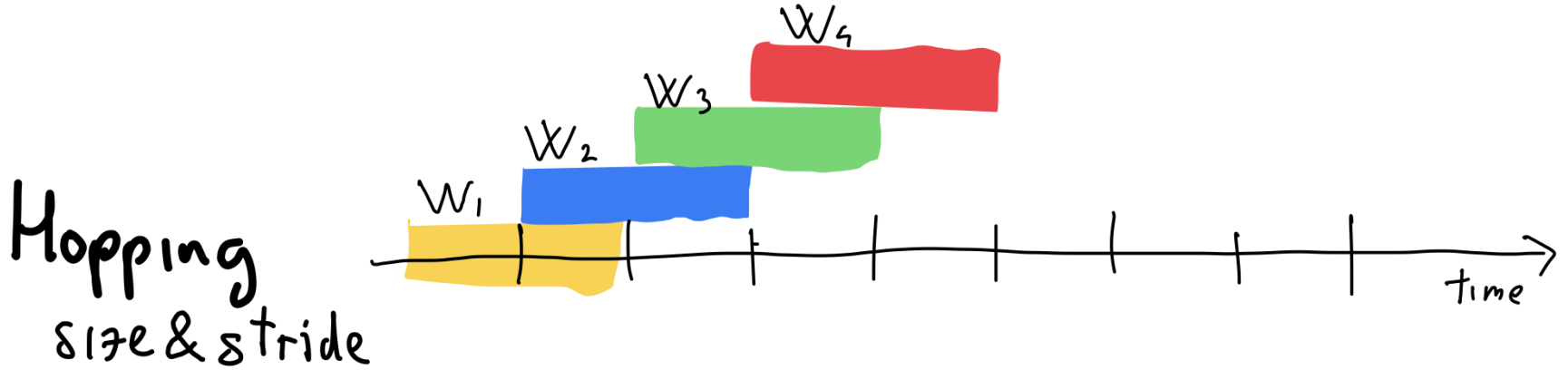
Introduction to Context in EPL

POLITECNICO
MILANO 1863

context enables more flexible and **stateful processing**
in scenarios requiring groupings or conditions
that go **beyond** simple event **windows**

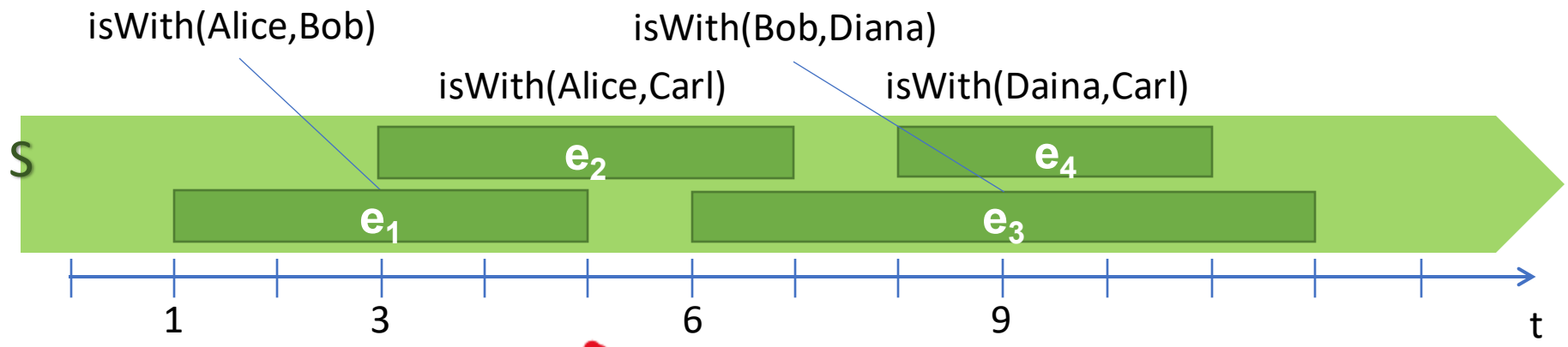
(The most important) Windows





recall

Interval-base Time Model



- We can ask the queries in the previous two slides
- It is possible to write even more complex queries:
  - Which are the meetings that last less than 5m?
  - Which are the meetings with **conflicts**?

joins(Alice,Bob)

splits(Alice,Bob)

recall

Why Use Context?

- **Better Control**: context gives fine-grained control over how events are processed
- **Stateful Processing**: handles cases where the state is important, **e.g., sessions** or transactions
- **Efficient Partitioning**: It helps reduce complexity in event partitioning by user, session, or custom logic

Types of Context

- **Context by Key**: Partitions events based on a key, e.g., userId
- **Context by Start/End Conditions**: Defined by specific events or triggers
- **Context by Time**: Defines temporal windows (generalizes logical windows, e.g. it allows to declare session windows)

Defining Context by Key

- Group all user actions to monitor their behavior

```
create context UserSessionContext
partition by userId from UserActionEvent;
```

- This context segments the event stream by userId
- Useful for handling stateful event processing tied to individual users

Stateful Event Processing with Context

POLITECNICO MILANO 1863

- E-commerce transactions, monitoring each order session

```
create context OrderContext
partition by userId from UserActionEvent
initiated by UserActionEvent(action='start_order')
terminated by UserActionEvent(action='complete_order’);
```

- Context starts when a user initiates an order and ends when she completes it
- Useful for tracking state transitions, like handling transactions or workflows

Defining Context by Time (and Key)

POLITECNICO MILANO 1863

- Time-based analysis such as tracking actions within fixed periods

```
create context TimeBatchContext
partition by userId from UserActionEvent
initiated by UserActionEvent
terminated after 10 minutes;
```

- This context processes events within 10-minute intervals
- It starts when a `UserActionEvent` occurs and ends after 10 minutes

Context in Action: Query Example

POLITECNICO MILANO 1863

- Calculating per-user metrics, like total purchases during a session.

```
context UserSessionContext
select avg(spentAmount) from UserActionEvent(action='purchase’);
```

- This allows per-session aggregation without losing the state across events.

Comparison: Context vs Windows

POLITECNICO MILANO 1863

Windows

- operate over a **fixed** time frame or number of events
- are stateless
- reset after each use

Contexts

- **provide more flexible segmentation**,
- have states that persist across event streams
- persist until terminated, providing a more robust grouping for complex scenarios

Comparison: Context vs Windows

POLITECNICO MILANO 1863

Windows

- operate over a **fixed** time frame or number of events
- are stateless
- reset after each use

Contexts

- **provide more flexible segmentation**,
- have states that persist across event streams
- persist until terminated, providing a more robust grouping for complex scenarios

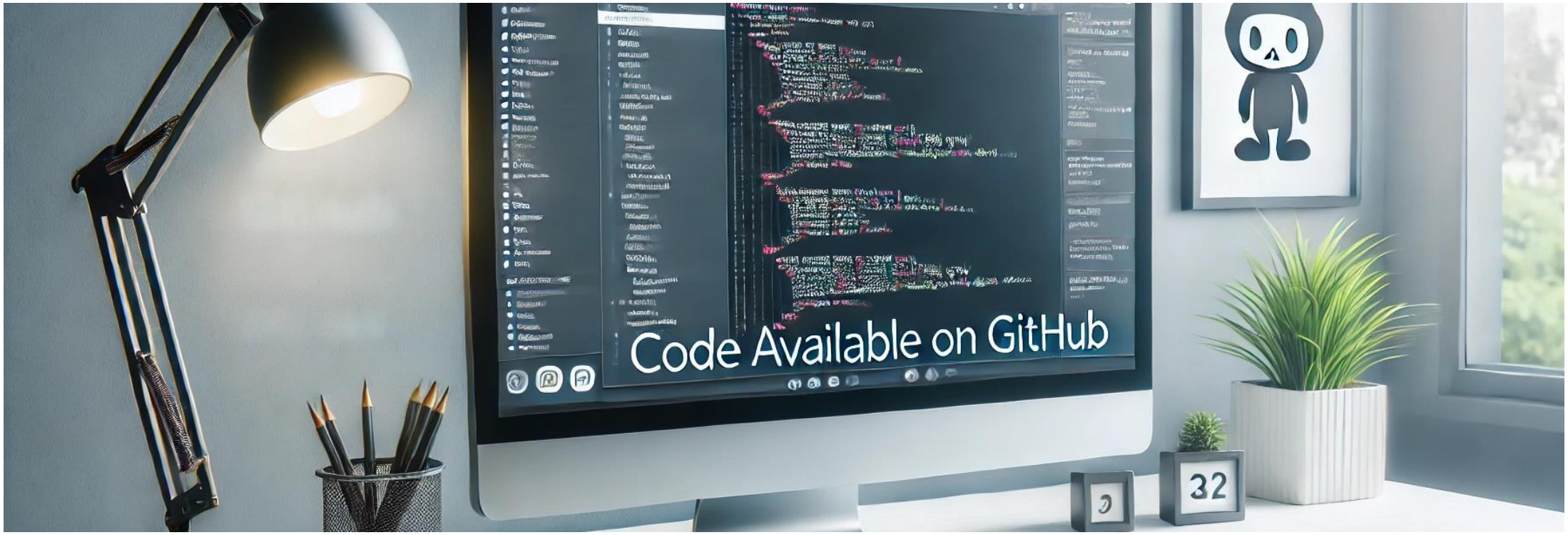
POLITECNICO
MILANO 1863

Use case: bocce game!

Code available on github!

- All the code described in this lecture is available at

  https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/EPL/epl_bocce



EPL and Esper
Contexts

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml

POLITECNICO
MILANO 1863